Refactoring Report: Deep Learning Teaching Codebase

Special Participation C — HW3 (Part 2)

Authors

Noah Lund Syrdal (3041928386)
Anders Vestrum (3041972833)
Srikar Babu (3041813297)

Department of Electrical Engineering and Computer Sciences
University of California, Berkeley

October 24, 2025
https://github.com/NoahLundSyrdal/SpecialParticipationC

1. Introduction

This report presents a detailed account of the refactoring process applied to a deep learning teaching codebase that was originally developed in a single Jupyter notebook. The initial version evolved organically to achieve the desired conceptual learning outcomes related to deep learning concepts such as scaling laws, μP, and optimizer behavior.

While pedagogically sound, the implementation lacked modularization, consistency, and adherence to Pythonic principles, making it difficult to maintain, test, or reuse.

The primary objective of this refactoring effort was to retain all teaching value and learning affordances of the original code while transforming it into a structured, maintainable, and professional codebase that follows good software and ML engineering practices. Following the principles of PEP 8 and PEP 257, as well as PyTorch's reproducibility guidelines, the code was separated into three main modules: `models`, `optimizers`, and `training` utilities. This new organization supports reproducibility, readability, and easier experimentation while maintaining conceptual continuity with the original material.

The complete refactored codebase is available on GitHub - Link. The repository also includes a solution notebook and supporting files, developed to produce a more optimal implementation while preserving the intent of the original version.

2. Project Structure and Modularization

Originally, all components—including model definitions, optimizer logic, and training routines—were interwoven within a single notebook file (`q_mup_coding.ipynb`). This structure limited scalability and hindered systematic experimentation.

To address these issues, the project was restructured into a new modular layout under a `src/` directory. The refactored hierarchy separates core functionalities into three dedicated Python modules—each responsible for a distinct logical component of the system.

```
src/
|-- models.py       # MLP, ScaledMLP (+ muP hooks)
|-- optimizers.py   # SimpleAdam, SimpleAdamMuP, SimpleShampoo
`-- training.py     # loops, LR sweeps, diagnostics & plots
```

This modularization enhances clarity and maintainability while facilitating code reuse across different experiments. By isolating definitions from usage, the design enables unit testing and better separation of concerns, mirroring standard practices in both software and ML engineering.

Note: The solution notebook `q_mup_coding_sol.ipynb` utilities files from `src_sol` folder, as opposed to `src` folder.

3. Models (models.py)

Both `MLP` and `ScaledMLP` maintain the same external interface: they accept `input_size`, `hidden_sizes`, and `num_classes` as arguments and return (logits, activations), where the first hidden activation is dropped and activations are detached. The μP-related experimental hook remains present between each Linear and Sigmoid layer, allowing students to explore scaling effects. The default PyTorch initialization is preserved, while optional schemes such as Xavier or Kaiming uniform can be enabled via an `init` argument.

From a software-engineering perspective, the module now includes descriptive docstrings and full type hints following PEP 257 and Google-style conventions. Constructor validation ensures valid input dimensions, and explicit error messages replace implicit assumptions. Layer creation and parameter initialization are centralized in helper methods (`_build_linears` and `reset_parameters()`), and a custom `__repr__` enables quick inspection of configuration.

From an ML-engineering perspective, the design supports reproducibility and safer experimentation. While Kaiming initialization is included for flexibility, Xavier remains the recommended choice for Sigmoid-based networks. Deterministic results require explicit seed-setting using `torch.manual_seed(...)`.

Overall, the refactored module is cleaner, more maintainable, and consistent with good engineering practices while preserving the conceptual behavior of the original notebook.

4. Optimizers (optimizers.py)

The optimizer implementations necessary for this homework (`SimpleAdam`, `SimpleAdamMuP`, `SimpleShampoo`, and `SimpleShampooScaled`) were moved into `src/optimizers.py`. This change separates implementation details from experiments and makes the notebook cleaner. The optimizers remain compatible with the PyTorch Optimizer API, and their mathematical update rules and arguments are unchanged.

Educational placeholders for μP scaling and Shampoo preconditioning remain, allowing students to experiment. Variable naming (e.g., `m`, `v`, `m_hat`, `v_hat`) matches the original math, keeping code–theory alignment intact.

The refactored code now validates hyperparameters, includes comprehensive docstrings, and ensures consistent parameter-state handling. From a style perspective, PEP 8 formatting and PEP 257 docstring standards improve readability and consistency.

From an engineering standpoint, the design is modular, testable, and easy to extend—supporting reproducible workflows without sacrificing educational clarity.

5. Training Utilities (training.py)

Training, evaluation, and diagnostic functions were refactored into `src/training.py`. This separation allows notebooks to focus on conceptual experiments while delegating procedural logic—such as training loops, activation tracking, and visualization—to a reusable module.

The utilities still use `CrossEntropyLoss`, handle model outputs as `(logits, activations)` with the first hidden activation dropped, and visualize activation drifts and parameter-update norms exactly as before. The TODO sections for μP and Shampoo remain preserved as instructional hooks.

From a software-engineering perspective, all public functions now include type hints and validation to catch errors early. Mutable defaults were removed, and device handling is unified via helper functions such as `get_device()` and `to_device()`. Seed management is centralized through `set_seed()`, and numerical safeguards (e.g., adding ϵ in RMS computations) prevent instability.

For deterministic GPU behavior, users should additionally set:

```
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

From an ML-engineering perspective, the utilities support deterministic learning-rate sweeps, stable metric averaging (mean over the final $k = 5$ validation losses), and modular visualization pipelines. These enhancements make the codebase both professional and extensible while preserving its instructional intent.

6. Summary

This refactoring transformed an ad-hoc educational notebook into a structured, maintainable software package. The modular design improves readability and reliability and adheres to best practices in software and ML engineering. Experiments are now easier to reproduce, extend, and maintain—without losing the original educational intent.

Before refactoring, the notebook lacked modularity, validation, and documentation. After refactoring, each module is well-documented, validated, and independently testable. Educational elements—such as activation drift analysis and optimizer dynamics—remain accessible and clear.

The refactor adheres to PEP 8 and PEP 257 conventions, NumPy-style docstrings, and PyTorch's reproducibility standards (`nn.Module`, `reset_parameters`, `torch.manual_seed`). It draws from initialization research by Glorot & Bengio (2010) and He et al. (2015), producing a codebase that combines pedagogical clarity with professional engineering quality.

7. Use of AI Tools

AI tools were employed in limited and transparent ways, consistent with academic integrity policies.

Code Refactoring

GitHub Copilot was used to assist with repetitive syntax patterns such as input validation, import management, and docstring formatting. All core algorithmic components, including the μP functionality, optimizer logic, and training routines, were implemented and verified manually.

Report Preparation

ChatGPT was used exclusively for stylistic refinement and grammatical consistency. No technical or conceptual content was generated by AI, and all explanations, analyses, and design decisions were written independently by the authors.

Summary

AI assistance was limited to code standardization and linguistic editing. All design choices, implementations, and reasoning remain entirely the work of the authors.